Programação de Aplicações Web

Aula 6: Introdução ao JavaScript, DOM e Web APIs

1. O que é JavaScript?

JavaScript é uma linguagem de programação de alto nível, interpretada e multiparadigma. É a "alma" da web, permitindo interatividade e dinamismo nas páginas que antes eram estáticas.

- **História**: Criada em 1995 por Brendan Eich (Netscape) em apenas 10 dias!
- **Ecossistema**: Hoje roda no navegador (Frontend) e no servidor (Backend com Node.js).
- **Padronização**: O nome oficial da linguagem é **ECMAScript (ES)**, e novas funcionalidades são lançadas anualmente.
- **Por que aprender?** É a linguagem mais popular do mundo e essencial para qualquer desenvolvedor web.

> "Qualquer aplicação que possa ser escrita em JavaScript, eventualmente será escrita em JavaScript." - Lei de Atwood

2. Variáveis e Constantes

No JavaScript moderno, usamos três palavras-chave para declarar recipientes de dados:

- `const` : Para valores que não mudam (constantes). **Sempre prefira** `const` .
- `let` : Para valores que podem ser reatribuídos. Possui escopo de bloco `{}` .
- `var` : A forma antiga. Evite usar pois possui comportamentos confusos de escopo (hoisting).

```
const nome = "Dev Web";
let contador = 0;

contador = contador + 1; // Permitido
// nome = "Novo Nome";   // Erro! Constantes não podem ser alteradas.

if (true) {
  let escopoLocal = "Visível apenas aqui dentro";
}
// console.log(escopoLocal); // Erro! escopoLocal não está definido aqui fora.
```

3. Tipos de Dados

O JavaScript é dinamicamente tipado, o que significa que você não precisa declarar o tipo explicitamente.

- **Primitivos**:
  - `String` : "Texto", 'Texto' ou `Texto com ${váriavel}` (Template Literals)
  - `Number` : 42, 3.14 (não há distinção entre int e float)
  - `Boolean` : `true` ou `false`
  - `null` : Representa a ausência intencional de valor.
  - `undefined` : Valor de uma variável declarada mas não inicializada.
- **Complexos**:
  - `Object` : `{ nome: "Paulo", idade: 30 }`
  - `Array` : `[1, 2, 3, "quatro"]`
  - `Function` : Blocos de código reutilizáveis.

4. Operadores

Ferramentas para manipular dados e realizar comparações.

- **Aritméticos**: `+`, `-`, `*`, `/`, `%` (resto), `**` (potência).
- **Comparação**:
  - == e !=: Compara valores (evite: faz conversão automática).
  - === e !==: Compara **valor e tipo** (sempre use estes).
  - >, <, >=, <=.
- **Lógicos**:
  - && (AND): Verdade se ambos forem verdade.
  - || (OR): Verdade se ao menos um for verdade.
  - ! (NOT): Inverte o valor booleano.

```
const idade = 20;
const temIngresso = true;

if (idade >= 18 && temIngresso) {
  console.log("Pode entrar no show!");
}
```

5. Estruturas Condicionais (if/else)

Permitem que o código tome caminhos diferentes baseados em condições booleanas.

- `if` : Executa o bloco se a condição for verdadeira.
- `else if` : Testa uma nova condição se a anterior for falsa.
- `else` : Executa se nenhuma condição anterior for verdadeira.

```
const hora = 14;

if (hora < 12) {
  console.log("Bom dia!");
} else if (hora < 18) {
  console.log("Boa tarde!");
} else {
  console.log("Boa noite!");
}
```

6. Estruturas Condicionais (switch)

Ideal quando temos uma única variável e queremos comparar com múltiplos valores possíveis.

- `case` : Define um valor de comparação.
- `break` : Para a execução do switch (importante: sem ele, o código "escorrega" para o próximo case).
- `default` : Funciona como o `else` .

```
const fruta = "Maçã";

switch (fruta) {
  case "Banana":
    console.log("Rica em potássio!");
    break;
  case "Maçã":
    console.log("Mantenha o médico longe!");
    break;
  default:
    console.log("Fruta desconhecida.");
}
```

7. Laços de Repetição (for)

Usados para repetir um bloco de código um número determinado de vezes.

- `for` **tradicional**: Útil quando sabemos exatamente quantas vezes repetir.
- `for ... of` : Perfeito para percorrer elementos de um array.

```
// For tradicional
for (let i = 0; i < 5; i++) {
  console.log(`Contagem: ${i}`);
}

// For ... of (mais comum no dia a dia)
const frutas = ["Maçã", "Banana", "Uva"];
for (const fruta of frutas) {
  console.log(fruta);
}
```

8. Laços de Repetição (while / do while)

Usados quando não sabemos exatamente o número de repetições, dependendo de uma condição dinâmica.

- `while` : Checa a condição **antes** de executar.
- `do while` : Executa o código ao menos **uma vez**, e depois checa a condição.

```
let contador = 0;

while (contador < 3) {
  console.log("Repetindo ... ");
  contador++;
}

do {
  console.log("Executo ao menos uma vez!");
} while (false);
```

9. O que é o DOM?

DOM (Document Object Model) é a representação do seu HTML como uma árvore de objetos que o JavaScript pode manipular.

Selecionando Elementos

Antigamente usávamos muitos IDs, hoje as APIs modernas facilitam tudo:

- `document.querySelector(".classe")` : Seleciona o primeiro elemento que bate com o seletor CSS.
- `document.querySelectorAll(".classe")` : Seleciona todos e retorna uma lista.
- `document.getElementById("meu-id")` : Seleção rápida por ID.

```
const titulo = document.querySelector("h1");
const botoes = document.querySelectorAll(".btn-primary");
```

10. Manipulação do DOM e Eventos

Uma vez selecionado, podemos mudar o que quisermos no elemento.

Alterando Conteúdo e Estilo

```
const titulo = document.querySelector("h1");
titulo.textContent = "Novo Título!"; // Muda apenas texto
titulo.style.color = "red";           // Muda CSS diretamente
```

Escutando Eventos

A forma mais comum de interagir com o usuário é o `addEventListener`.

```
const botao = document.querySelector("#meu-botao");

botao.addEventListener("click", () => {
  console.log("O botão foi clicado!");
  alert("Olá mundo!");
});
```

11. Fazendo Requisições (Fetch API)

Páginas modernas raramente trabalham sozinhas. Elas buscam dados de APIs.

- `fetch()` : Retorna uma **Promise** (um valor que estará disponível no futuro).
- `async/await` : A forma mais legível de trabalhar com código assíncrono.

```
async function buscarDados() {
  try {
    const resposta = await fetch("https://api.github.com/users/paulormnas");
    const dados = await resposta.json();
    console.log(`Nome no GitHub: ${dados.name}`);
  } catch (erro) {
    console.error("Erro ao buscar dados:", erro);
  }
}

buscarDados();
```

12. Rodando e Debugando no Navegador

Todo navegador moderno tem ferramentas incríveis para desenvolvedores (F12 ou Ctrl+Shift+I).

- **Console**: Para testar pequenos trechos de código e ver logs (`console.log`).
- **Sources (Fontes)**: Onde a mágica acontece.
  - **Breakpoints**: Clique na linha de código para pausar a execução.
  - **Watch**: Monitore o valor de variáveis em tempo real enquanto o código roda.
  - **Step Over/Into**: Caminhe linha a linha para entender a lógica.

> "Se você não está usando o Debugger das DevTools, você está programando no escuro."

13. JavaScript vs TypeScript

TypeScript é um "supeconjunto" do JavaScript que adiciona **Tipagem Estática**.

Característica	JavaScript	TypeScript
Tipagem	Dinâmica (variável muda de tipo)	Estática (você define o tipo)
Erros	Descobertos ao rodar (Runtime)	Descobertos ao escrever (Compile-time)
Produtividade	Rápido para protótipos	Melhor para projetos grandes

```
// Em TypeScript
function somar(a: number, b: number): number {
  return a + b;
}

somar(5, "10"); // O editor te avisa do erro na hora!
```

14. Conclusão e Próximos Passos

Hoje vimos o básico necessário para tornar as páginas web interativas.

- Sintaxe básica e variáveis.
- Lógica de programação (condicionais e loops).
- Manipulação do DOM para mudar o que o usuário vê.
- Requisições assíncronas com Fetch.

🚀 Desafio Prático:

Tente selecionar um botão do seu HTML, adicione um clique, busque um dado em uma API pública (como a PokéAPI ou JSONPlaceholder) e exiba esse dado em um parágrafo da tela.

Até a próxima aula! 👋